

Ein Handbuch zu Datalog

Historische Einordnung und Einführung
Claude.AI (Opus 4.8, hoch)

2026-07-14

Inhalt

1. Was Datalog ist — in einem Absatz	3
Teil I — Historische und aktuelle Einordnung	4
Teil II — Datalog als Sprache	6
Teil III — Programmieren mit Datalog	8
Teil IV — Grenzen, Semantik-Fallstricke und Ausblick	13
Anhang — Referenz der Engine und Ausführung	15

1. Was Datalog ist — in einem Absatz

Datalog ist eine deklarative Logik- und Datenbanksprache. Ein Programm besteht aus **Fakten** (bekannten Grunddaten) und **Regeln** (die aus Bekanntem Neues ableiten); eine **Anfrage** liest die abgeleiteten Ergebnisse aus. Anders als SQL beherrscht Datalog **Rekursion** von Haus aus — die transitive Hülle eines Graphen etwa lässt sich in einer Zeile ausdrücken. Anders als die vollständige Logikprogrammierung (Prolog) verzichtet Datalog auf Funktionssymbole und garantiert dadurch, dass die Auswertung terminiert und ein eindeutiges Ergebnis besitzt. Man beschreibt *was* gelten soll, nicht *wie* es zu berechnen ist; um die Berechnung kümmert sich die Engine.

Teil I — Historische und aktuelle Einordnung

Wurzeln: Logik trifft Datenbanken (1970er)

Datalog entstand am Zusammenfluss zweier Strömungen. Die eine ist die **Logikprogrammierung**: Anfang der 1970er formulierten Alain Colmerauer und Robert Kowalski die Idee, Hornklauseln als Programme zu lesen — daraus wurde Prolog. Die andere ist die **relationale Datenbanktheorie**: 1970 formalisierte Edgar F. Codd Datenbanken über relationale Algebra und relationalen Kalkül. Beide Welten trafen sich in den **deduktiven Datenbanken** — Datenbanksystemen mit Schlussfolgerungsfähigkeit —, deren Paradigma unter anderem der von Jack Minker herausgegebene Band *Logic and Data Bases* (1978) prägte.

Der Antrieb war eine konkrete Lücke: relationale Algebra und Kalkül können einfache, aber unentbehrliche Operationen wie die transitive Hülle eines Graphen *nicht* ausdrücken. Genau diese Rekursion liefert Datalog.

Der Name und die Formalisierung (1980er)

Der Begriff „Datalog“ wurde in den 1980er-Jahren geprägt und wird dem Datenbankforscher David Maier zugeschrieben (in manchen Quellen gemeinsam mit David S. Warren genannt). Technisch ist Datalog ein **funktionssymbolfreier Ausschnitt von Prolog**: Terme sind nur Konstanten und Variablen. Diese Einschränkung ist der entscheidende Kunstgriff — sie macht die **Bottom-up-Auswertung** (von den Fakten aufwärts bis zum Fixpunkt) berechenbar und terminierend, im Gegensatz zu Prologs Top-down-Suche, die in Endlosschleifen laufen kann.

Theoretische Meilensteine

In den 1980ern entstand das begriffliche Fundament, auf dem heutige Engines — auch die begleitende — noch stehen:

- **Immediate-Consequence-Operator und kleinster Fixpunkt.** Die Semantik eines positiven Programms ist der kleinste Fixpunkt eines monotonen Ableitungsoperators (in der Tradition von van Emden und Kowalski). Bottom-up-Auswertung berechnet ihn konstruktiv.
- **Semi-naive Auswertung.** Eine Optimierung, die pro Iteration nur mit den *neu* hinzugekommenen Fakten arbeitet, statt alles neu zu berechnen.
- **Magic Sets (1986).** Bancilhon, Maier, Sagiv und Ullman zeigten, wie man ein Programm so umschreibt, dass die Bottom-up-Auswertung nur die für eine konkrete Anfrage *relevanten* Fakten erzeugt — bedarfsgesteuerte Auswertung auf großen Datenbeständen.
- **Stratifizierte Negation (Mitte/Ende der 1980er).** Um Negation sinnvoll zu erlauben, ordnet man Prädikate in Schichten (Strata) an, sodass ein negiertes Prädikat stets vollständig berechnet ist, bevor es benutzt wird.
- **Well-founded-Semantik (1991) und Stable-Model-Semantik (1988)** erweitern die Bedeutung auf Programme, die sich *nicht* schichten lassen; letztere begründete das heutige Answer-Set-Programming.

Als Standardreferenz dieser Ära gilt Ullmans *Principles of Database and Knowledge-Base Systems* (1988/89).

Niedergang und Wiederaufstieg

In den 1990ern erlahmte das Interesse. Die Gründe waren weniger theoretischer als praktischer Natur: unreife Hardware, die Rigidität früher Systeme und der Umstand, dass sich deduktive Datenbanken kommerziell nicht durchsetzten. Datalog blieb ein akademisches Thema.

Seit den 2000ern erlebt die Sprache eine Renaissance — getrieben nicht von der Datenbankwelt, sondern von neuen Anwendungsfeldern, in denen „berechne alle Konsequenzen eines Regelwerks“ genau die richtige Abstraktion ist.

Aktuelle Bedeutung

Datalog ist heute wieder ein aktives Feld, sowohl in der Forschung als auch in produktiven Systemen. Die wichtigsten Stränge:

- **Statische Programmanalyse.** Der prominenteste Anwendungsfall. Die Engine **Soufflé** übersetzt Datalog-Regeln in hochoptimierten C++-Code und erreicht damit die Leistung handgeschriebener Analysen; das Doop-Framework etwa formuliert Points-to-Analysen für große Codebasen als Datalog-Programme.
- **Wissensgraphen und Ontologien.** Systeme wie **RDFOx** und **Vadalog** nutzen Datalog als Schlussmaschine über RDF-Daten; Fragmente von OWL 2 RL lassen sich direkt in Datalog-Regeln übersetzen.
- **Datenbanken mit Datalog-Anfragesprache.** **Datomic** (im Clojure-Ökosystem), **XTDB**, **DataScript** und **CozoDB** bieten Datalog-artige Abfragen; **Logica** (Google) und kommerzielle Systeme wie RelationalAI setzen Datalog als Schicht über SQL- bzw. Spalten-Datenbanken.
- **Anhaltende Forschungsdynamik.** Neben Engines wie **Nemo** (TU Dresden) und Erweiterungen um Gitter und SMT (Flix, Datafun, Formulog) erscheinen laufend neue Arbeiten zu Auswertung und Compilern (etwa Flan, FlowLog, GPU-basierte Datalog-Auswertung, 2024–2025). Überblicke geben die Monografie *Modern Datalog Engines* (Ketsman & Kourtis) und Krötzschs *Modern Datalog: Concepts, Methods, Applications*.

Kurz: Datalog wird dort geschätzt, wo man **rekursive Ableitungen über strukturierten Daten deklarativ, wartbar und effizient** ausdrücken will — von Compiler-Analysen über Zugriffskontrolle bis zu Wissensgraphen.

Teil II — Datalog als Sprache

Dieser Teil führt die Sprache systematisch ein. Die Syntax ist die der begleitenden Engine; sie folgt der akademischen Konvention (Prolog-nah), mit `not` für Negation.

2.1 Grundbausteine

- **Konstante** — ein konkreter Wert. In der Engine: ein kleingeschriebener Bezeichner (`alice`, `bob`), eine Zahl (`1`, `3.5`) oder ein String (`"Text"`).
- **Variable** — ein Platzhalter. Beginnt mit Großbuchstaben oder Unterstrich (`X`, `Y`, `_N`).
- **Atom** — ein Prädikat, angewandt auf Terme: `parent(alice, bob)`, `edge(X, Y)`. Die Stelligkeit (Arität) ist die Anzahl der Argumente.
- **Grundatom** — ein Atom ohne Variablen, also nur mit Konstanten.
- **Fakt** — ein Grundatom, das als wahr gesetzt wird: `parent(alice, bob)`.
- **Regel** — Kopf `:-` Rumpf. Der Rumpf ist eine kommagetrennte Konjunktion von Literalen. Gelesen: „Der Kopf gilt, *wenn* alle Rumpf-Literale gelten.“
- **Literal** — ein positives Atom oder ein negiertes (`not p(X)`) oder ein eingebauter Vergleich (`X < Y`).
- **Anfrage** — ein Rumpf, den man in der REPL stellt: `ancestor(alice, X)`. Konstanten schränken ein, Variablen sammeln Ergebnisse. Derselbe Variablenname an mehreren Stellen wirkt als impliziter Gleichheitsfilter (ein Join).

Ein Kommentar beginnt mit `//`, jede Klausel endet mit einem Punkt.

2.2 Semantik: EDB, IDB und Fixpunkt

Die Daten zerfallen in zwei disjunkte Teile:

- **EDB (Extensional Database)** — die Prädikate, die *nur* als Fakten vorkommen und nie im Kopf einer Regel stehen. Das sind die Eingabedaten.
- **IDB (Intensional Database)** — die Prädikate, die im Kopf mindestens einer Regel stehen. Ihre Fakten werden *abgeleitet*.

Die Bedeutung eines positiven Programms ist sein **kleinster Fixpunkt**: Man startet mit der EDB und wendet alle Regeln wiederholt an, bis keine neuen Fakten mehr entstehen (Sättigung). Weil die Regeln monoton sind und keine Funktionssymbole neue Werte erfinden, ist dieser Prozess garantiert endlich. Die begleitende Engine berechnet ihn *naiv*: In jeder Runde werden alle Regeln erneut ausgewertet, bis eine Runde nichts Neues mehr liefert.

2.3 Rekursion

Ein Prädikat darf im Kopf *und* im Rumpf einer Regel vorkommen. Das ist der Kern von Datalogs Ausdruckskraft. Das kanonische Muster ist die transitive Hülle:

```
reach(X, Y) :- edge(X, Y).           // Basisfall
reach(X, Y) :- edge(X, Z), reach(Z, Y). // rekursiver Fall
```

2.4 Negation, Stratifizierung und Sicherheit

Ein negiertes Literal `not p(...)` gilt, wenn das entsprechende Grundatom **nicht** ableitbar ist (Negation als Fehlschlag, unter der Annahme einer geschlossenen Welt). Damit das wohldefiniert ist, gelten zwei Bedingungen:

- **Stratifizierbarkeit.** Negation darf nicht in einem rekursiven Zyklus auftreten. Formal: Kein negiertes Prädikat darf — direkt oder indirekt — von der Regel abhängen, die es negiert. Die Engine ordnet die Prädikate in Strata; ein negiertes Prädikat liegt stets in einem echt niedrigeren Stratum und ist daher fertig berechnet, bevor der Anti-Join darauf zugreift. Zyklische Negation wird abgelehnt.
- **Sicherheit (Safety).** Jede Variable, die im Kopf oder in einem negierten Literal vorkommt, muss auch in einem positiven Rumpf-Literal auftreten. Dieses positive Literal „verankert“ die Variable an einer endlichen Menge bekannter Werte. Ohne diese Bedingung wäre `not p(X)` gleichbedeutend mit „alle X des Universums außer p“ — nicht berechenbar. Unsichere Regeln werden abgelehnt.

Beim Ausführen zeigt die Engine die berechnete Schichtung an, etwa `Strata: {abnormal=0, bird=0, flies=1, penguin=0}` — `flies` liegt eine Ebene über dem negierten `abnormal`.

2.5 Eingebaute Vergleiche und Arithmetik

Über die reine Logik hinaus bietet die Engine Vergleiche `= != < <= > >=` und Arithmetik `+ - * / %` innerhalb von Vergleichen. Das `=` wirkt dabei auch als **Binder**: Steht auf einer Seite eine noch ungebundene Variable und ist die andere Seite auswertbar, wird gebunden — etwa `M = N + 1`. Die Division `/` ist Fließkomma-Division (`7 / 2` ergibt `3.5`); geht sie auf, bleibt das Ergebnis ganz.

Teil III — Programmieren mit Datalog

Nun die Praxis, in aufsteigender Komplexität. Jedes Beispiel besteht aus einem Programm (im Ordner `examples/`), einer oder mehreren REPL-Anfragen und der *tatsächlich erzeugten* Ausgabe. Gestartet wird jeweils mit

```
java Datalog.java examples/<datei>.dl
```

und danach tippt man die Anfrage in die REPL (Prompt `?-`).

3.1 Fakten, Joins und Rekursion — eine Verwandtschaftsdatenbank

`examples/family.dl`:

```
parent(alice, bob).
parent(alice, carol).
parent(bob, dave).
parent(carol, eve).
parent(dave, frank).

grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
sibling(X, Y)      :- parent(P, X), parent(P, Y), X != Y.

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Die Regel für `grandparent` ist ein **Self-Join**: Die gemeinsame Variable `Z` verknüpft zwei `parent`-Fakten. `sibling` nutzt zusätzlich den eingebauten Vergleich `X != Y`, um die trivialen Paare (jeder ist mit sich selbst „verwandt“) auszuschließen. `ancestor` ist die rekursive transitive Hülle von `parent`.

```
?- grandparent(alice, X).
  X = dave
  X = eve
2 solution(s).

?- sibling(bob, X).
  X = carol
1 solution(s).

?- ancestor(alice, X).
  X = bob
  X = carol
  X = dave
  X = eve
  X = frank
5 solution(s).
```

Anfragen lassen sich in beide Richtungen stellen — man kann auch nach den *Vorfahren* eines Knotens fragen, indem man die zweite Stelle bindet:


```
?- ancestor(X, frank).
   X = dave
   X = bob
   X = alice
3 solution(s).
```

3.2 Graphen: Erreichbarkeit, Zyklen und Senken

examples/graph.dl (ein Graph mit 3-Zyklus $a \rightarrow b \rightarrow c \rightarrow a$ und Anhang $c \rightarrow d$):

```
node(a). node(b). node(c). node(d).
edge(a, b). edge(b, c). edge(c, a). edge(c, d).

path(X, Y) :- edge(X, Y).
path(X, Y) :- edge(X, Z), path(Z, Y).

in_cycle(X) :- path(X, X).           // ein Knoten, der sich selbst erreicht
has_out(X)  :- edge(X, Y).
sink(X)     :- node(X), not has_out(X).
```

Drei nützliche Muster auf einen Streich. `path` ist die Erreichbarkeit. `in_cycle` findet Knoten in einem Zyklus, elegant über die Selbst-Erreichbarkeit `path(X, X)`. `sink` nutzt **Negation als Filter**: ein Knoten ohne ausgehende Kante. Man beachte die Sicherheit — `node(X)` verankert `X`, bevor `not has_out(X)` es filtert.

```
?- path(a, X).
   X = b
   X = c
   X = a
   X = d
4 solution(s).

?- in_cycle(X).
   X = a
   X = b
   X = c
3 solution(s).

?- sink(X).
   X = d
1 solution(s).
```

`a` erreicht sich selbst über den Zyklus und taucht daher in `path(a, X)` auf; `d` liegt außerhalb des Zyklus und ist die einzige Senke.

3.3 Negation für Defaults mit Ausnahmen (nichtmonotones Schließen)

Das klassische Lehrbuchbeispiel: Vögel fliegen — außer den anomalen (hier: Pinguinen).

examples/defaults.dl:

```
bird(tweety). bird(polly). bird(pingu).
penguin(tweety). penguin(pingu).

abnormal(X) :- penguin(X).
flies(X)    :- bird(X), not abnormal(X).
```

Das ist die typische **Default-Regel mit Ausnahmeliste**: Die Ausnahme wird in einem eigenen Prädikat (`abnormal`) gesammelt und im Default negiert. Weil `flies` das negierte `abnormal` benutzt, landet es genau ein Stratum höher.

```
?- flies(X).
   X = polly
1 solution(s).

?- abnormal(X).
   X = tweety
   X = pingu
2 solution(s).
```

Nur `polly` fliegt; `tweety` und `pingu` sind als Pinguine anomal. Dieses Muster — Regel plus negierte Ausnahme — ist der Standardweg, in Datalog Voreinstellungen mit Ausnahmen auszudrücken.

3.4 Vergleiche und Arithmetik

`examples/arithmetic.dl`:

```
num(1). num(2). num(3). num(4). num(5). num(6).

even(N)      :- num(N), 0 = N % 2.
odd(N)       :- num(N), not even(N).
double(N, M) :- num(N), M = N * 2.
big(N)       :- num(N), N >= 4.
between(N)   :- num(N), N > 1, N < 5.
```

`even` filtert über den Rest modulo 2. `odd` ist schlicht die Negation von `even` (und liegt daher ein Stratum höher). `double` zeigt das `=` als **Binder**: `M` ist zunächst ungebunden und wird an `N * 2` gebunden. `big` und `between` demonstrieren Ordnungsvergleiche.

```

?- even(N).
  N = 2
  N = 4
  N = 6
3 solution(s).

?- odd(N).
  N = 1
  N = 3
  N = 5
3 solution(s).

?- double(N, M).
  N = 1, M = 2
  N = 2, M = 4
  N = 3, M = 6
  N = 4, M = 8
  N = 5, M = 10
  N = 6, M = 12
6 solution(s).

?- between(N).
  N = 2
  N = 3
  N = 4
3 solution(s).

```

Man kann Ausdrücke auch direkt auswerten lassen — eine Anfrage ganz ohne Datenbank:

```

?- X = 2 * 3 + 1.
  X = 7
1 solution(s).

```

3.5 Mehrschichtige Stratifizierung

Ein Beispiel, in dem sich drei Ebenen stapeln: `safe` (aus der EDB), darüber das negierte `unsafe`, darüber die rekursive, wiederum negativ gefilterte `reach`.

`examples/strata.dl`:

```

approved(1). approved(2). approved(3).
node(1). node(2). node(3). node(4).
edge(1, 2). edge(2, 3). edge(3, 4). edge(1, 3).

safe(X)      :- approved(X).
unsafe(X)    :- node(X), not safe(X).
reach(X, Y)  :- edge(X, Y), not unsafe(Y).
reach(X, Z)  :- reach(X, Y), edge(Y, Z), not unsafe(Z).

```

Beim Laden meldet die Engine die Schichtung `Strata: {approved=0, edge=0, node=0, reach=2, safe=0, unsafe=1}`. `safe` und die EDB liegen in Stratum 0, `unsafe` (negiert `safe`) in Stratum 1, `reach` (negiert `unsafe`) in Stratum 2. `reach` sammelt nur Pfade, deren Schritte auf sichere Knoten führen — der Schritt auf den unsicheren Knoten 4 wird verworfen.

```
?- unsafe(X).  
   X = 4  
1 solution(s).  
  
?- reach(1, Y).  
   Y = 2  
   Y = 3  
2 solution(s).
```

Weil `unsafe` vollständig berechnet ist, bevor `reach` es abfragt, „sieht“ der Anti-Join stets die komplette Menge — genau das ist die Garantie, die die Stratifizierung liefert.

Teil IV — Grenzen, Semantik-Fallstricke und Ausblick

4.1 Wenn ein Programm abgelehnt wird

Zwei Klassen von Programmen sind bewusst *keine* gültigen Datalog-Programme, und eine gute Engine weist sie mit einer klaren Diagnose ab, statt ein falsches oder undefiniertes Ergebnis zu liefern.

Rekursive Negation ist nicht stratifizierbar. `examples/reject_cycle.dl`:

```
q(1).  
p(X) :- q(X), not r(X).  
r(X) :- q(X), not p(X).
```

`p` und `r` negieren einander und bilden einen Zyklus über eine Negationskante. Es gibt kein eindeutiges Modell (die Belegungen würden endlos oszillieren). Die Engine lehnt ab:

```
Program is not stratifiable: negation occurs inside a recursive cycle.
```

Unsichere Regeln verletzen die Domänenunabhängigkeit. `examples/reject_unsafe.dl`:

```
paid(alice).  
broke(X) :- not paid(X).
```

`X` erscheint nur in einem negierten Literal, ohne positiven Anker. Das entspräche „alle `X` des Universums außer den bezahlten“ — unendlich. Die Engine lehnt ab:

```
Program error: Unsafe rule (head variable X unbound): broke(X) :- not paid(X).
```

Beides sind keine Schwächen, sondern Merkmale: Die Sprache tauscht ein Stück Ausdrucksmacht gegen die Garantie, dass jedes akzeptierte Programm terminiert und eine eindeutige Bedeutung hat.

4.2 Grenzen der Ausdrucksstärke

Reines Datalog erfasst nur monotone Ableitungen; „echte“ Nichtmonotonie (Negation) kommt erst mit Stratifizierung hinzu, und selbst dann bleiben nicht-stratifizierbare Programme außen vor — für sie braucht es die Well-founded- oder Stable-Model-Semantik (Answer-Set-Programming). Aggregate (SUM, COUNT, MIN) sind in Standard-Datalog nicht enthalten; sie sind, wie Negation, nichtmonoton und erfordern dieselbe Sorgfalt in der Auswertungsreihenfolge. Die begleitende Engine deckt bewusst den stratifizierten Kern samt eingebauter Vergleiche ab.

4.3 Effizienz und der Weg zu Magic Sets

Die naive Auswertung berechnet die *komplette* Ableitung, unabhängig von der Anfrage. Bei kleinen Datenbeständen oder Anfragen, die ohnehin „alles“ wollen, ist das optimal. Ihre Grenze zeigt sich erst bei **großen, dünnen Daten mit selektiven, gebundenen Anfragen**: Dann berech-

net die naive Engine viel, das die Anfrage gar nicht braucht. Hier setzt die **Magic-Sets-Transformation** an — sie schreibt das Programm so um, dass nur die für die Anfrage relevanten Fakten entstehen, und verwandelt bei Erreichbarkeitsanfragen quadratischen in linearen Aufwand. Die Kombination von Magic Sets mit stratifizierter Negation ist überraschend subtil (verfrühte Anti-Joins).

4.4 Ausblick

Wer über den stratifizierten Kern hinaus will, findet natürliche nächste Schritte in semi-naiver Auswertung (Effizienz), Aggregaten, gut-fundierter bzw. stabiler Semantik (nicht-stratifizierbare Programme) und eben Magic Sets (bedarfsgesteuerte Auswertung). Jeder dieser Schritte ist gut untersucht; die im Teil I genannten Überblicksarbeiten sind dafür der beste Einstieg.

Anhang — Referenz der Engine und Ausführung

A.1 Syntax auf einen Blick

Element	Schreibweise	Beispiel
Konstante (symbolisch)	kleingeschrieben	alice, edge
Konstante (Zahl)	Ganzzahl / Dezimal	1, 3.5
Konstante (String)	in Anführungszeichen	"Text"
Variable	Großbuchstabe / <code>_</code>	X, <code>_N</code>
Fakt	Grundatom + <code>.</code>	parent(alice, bob).
Regel	Kopf <code>:-</code> Rumpf.	p(X) <code>:-</code> q(X), r(X).
Negation	<code>not</code> vor dem Atom	not blocked(Y)
Vergleich	<code>=</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	X <code>>=</code> 4
Arithmetik	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (in Vergleichen)	M = N + 1
Kommentar	<code>//</code> bis Zeilenende	<code>//</code> ein Kommentar
Anfrage (REPL)	Rumpf + <code>.</code>	ancestor(alice, X).

Konvention: Groß = Variable, klein = Konstante/Prädikat. `/` ist Fließkomma-Division, `%` ist Modulo. Regeln sind mengenbasiert; die Reihenfolge der Rumpf-Literale ist für das Ergebnis unerheblich (die Engine wertet erst positive Joins, dann `=`-Bindungen und Vergleiche, zuletzt Negation aus).

A.2 REPL-Kommandos

Kommando	Wirkung
<code><anfrage>.</code>	Anfrage auswerten und Lösungen zeigen
<code>:load <datei></code>	Programm laden (ersetzt das aktuelle)
<code>:add <klausel></code>	Fakt/Regel ergänzen und neu auswerten
<code>:facts [pred]</code>	abgeleitete Fakten auflisten
<code>:rules</code>	aktuelles Programm anzeigen
<code>:strata</code>	berechnete Schichtung anzeigen
<code>:help</code>	Hilfe
<code>:quit</code>	REPL verlassen

A.3 Ausführung

Mit einer JRE genügt der Einzeldatei-Start (kein separates Kompilieren nötig):

```
java Datalog.java examples/family.dl
?- ancestor(alice, X).
```

Mit einem JDK alternativ:

```
javac Datalog.java
java Datalog examples/family.dl
```

Alle Beispiele lassen sich zudem im Stapel als Selbsttest prüfen — die Engine kennt einen `--test`-Modus, der eingebettete `//test`-Direktiven ausführt und Lösungsmengen ordnungsunabhängig vergleicht:

```
java Datalog.java --test examples/*.dl
```

A.4 Verzeichnis der Beispiele

Datei	Thema	zentrale Anfrage
family.dl	Fakten, Joins, Rekursion, Ungleichheit	ancestor(alice, X).
graph.dl	Erreichbarkeit, Zyklen, Senken (Negation)	sink(X).
defaults.dl	Defaults mit Ausnahmen (Nicht-monotonie)	flies(X).
arithmetic.dl	Vergleiche und Arithmetik	double(N, M).
strata.dl	mehrschichtige Stratifizierung	reach(1, Y).
reject_cycle.dl	rekursive Negation → abgelehnt	(Laden)
reject_unsafe.dl	unsichere Regel → abgelehnt	(Laden)

A.5 Quellen und weiterführende Literatur

- S. Ceri, G. Gottlob, L. Tanca: *What You Always Wanted to Know About Datalog (And Never Dared to Ask)*, IEEE TKDE, 1989 — klassische Einführung.
- J. D. Ullman: *Principles of Database and Knowledge-Base Systems*, Vol. II, 1989 — Standardreferenz zu Auswertung, Magic Sets und Stratifizierung.
- F. Bancilhon, D. Maier, Y. Sagiv, J. D. Ullman: *Magic Sets and Other Strange Ways to Implement Logic Programs*, PODS 1986.
- A. Van Gelder, K. Ross, J. Schlipf: *The Well-Founded Semantics for General Logic Programs*, JACM 1991.
- M. Abiteboul, R. Hull, V. Vianu: *Foundations of Databases*, 1995 — Kapitel zu Datalog und seiner Theorie.
- „Datalog: concepts, history, and outlook“ (2018) — moderner Rückblick samt Wiederaufstieg und aktuellen Systemen.

- B. Ketsman, P. Koutris: *Modern Datalog Engines*, Foundations and Trends in Databases, 2022; M. Krötzsch: *Modern Datalog: Concepts, Methods, Applications* (Reasoning Web, 2024/2025).
 - Systeme zum Ausprobieren: Soufflé (Programmanalyse), RDFox (Wissensgraphen), Nemo, Datomic / XTDB / CozoDB (Datalog-Anfragesprache in Datenbanken).
-

Dieses Handbuch begleitet eine kompakte, naiv auswertende Datalog-Engine mit stratifizierter Negation und eingebauten Vergleichen. Sämtliche gezeigten Ergebnisse wurden mit dieser Engine erzeugt und geprüft.